



The QMdbSim2 Project

API Developer Documentation

Document History

2026.07.10 – Initial documentation. Caveat emptor.

Prerequisites/Caveats

If you haven't already done so, please start with the QMdbSim2 Overview, Basic User, and Device Developer PDFs.

For API development, you will need basic Java skills, hopefully significantly more than I have. Be aware that my Java code will probably abuse Java naming and coding conventions. And when I speak of “packages,” “interfaces,” “classes,” etc., I'm likely not using the words in the strict Java sense. I trust that context will suffice to eliminate confusion.

I used Claude.AI with this project out of curiosity. While Claude works reasonably well, I have not yet done a full “code review” (restructuring, refactoring, etc.). In particular, *do not assume that the code comments are correct*.

Finally, this document provides minimal information to get you started: A small bit about the Microchip software simulator, generally where to find relevant code, and some tips. For the details, well, the code is the documentation.

Overview

MDBCCore

The Microchip simulator is implemented in the Java MDBCCore files installed automatically with MPLabX. The IDE uses it to communicate with hardware devices (PICKit, ICE, etc.) to program devices, run debugging sessions with those devices, etc. The software simulator is treated as just another hardware device. (As noted elsewhere, the software simulator does not implement all hardware functionality for all devices.)

The MDBCCore API is complex. The SDK provides a simplified API, MDB Core Scripting (MdbCS). This is included in the SDK as `mdbcs.java` and contains a `Debugger` class that does pretty much all that we need.

The QMdbSim2 project wraps the MdbCS code in a QMdbCS class which (primarily) combines multiple operations into single method calls to reduce DLL-to-Java JNI overhead.¹

The Java_API project folder contains the MdbCS/QMdbCS code as well as a minimal subset of the SDK sufficient to compile the QMdbCS.jar using Ant. See the Resources section below for a link to the full SDK. (As of this writing, we're using v6.30 of the SDK and IDE.)

Note: The SDK specifies that MDBCORE was compiled with Java 8....

DLL Code

As described in the Device Developer PDF, DLL code contains device-specific code (i.e., contains the evaluation function, e.g., ATTinyx5.cpp) compiled/linked with shared generic code (QMdbSim2_Shared). The shared code library communicates with QMdbCS.jar via JNI calls.

Key files:

MdbConfig.h.cpp — Loads QMdbCfg.ini and derives all JVM/MDBCS paths from three ini-specified roots (MplabxRoot, JdkRoot, QMdbcsJar). Resolves the jar path relative to the ini directory when not absolute, validates that jvm.dll is 32-bit via PE header inspection (since QSpice is a 32-bit process), and confirms existence of all derived paths.

JvmHost.h.cpp — Loads and starts an embedded 32-bit JVM via JNI_CreateJavaVM, dynamically resolving jvm.dll and its dependencies. Builds the MDBCS classpath by scanning MPLAB X module directories for JARs, detects the JVM major version to conditionally apply --add-opens (Java 9+ only), and suppresses NetBeans platform logging noise.

MdbcsDebugger.h.cpp — Thin JNI wrapper mapping 1:1 to com.microchip.mdbcs.QMdbCS. Caches method IDs during construct(), exposes the debugger lifecycle (connect / program / step / reset / disconnect / destroy), pin lookup with global-ref ownership, and an aggregated single-call getPinState() that unpacks a bit-packed long (voltage as float bits, I/O and analog/digital flags) for performance.

MdbSim.h.cpp — Public-facing simulator API replacing the old stdio-pipe QMdbSim, built on MdbcsDebugger/JvmHost. Manages explicit init state (NotStarted → Initialised → Running → Stopped), VDD sequencing, and bulk pin port mapping (PinPortMap) for QSpice C-Block integration, including input-voltage change detection to skip redundant JNI calls.

PinState.h — Header-only shared struct (PinState) plus pin-state constants (analog/digital, input/output), extracted to break the circular include between MdbSim.h and MdbcsDebugger.h.

I will be reworking the code so the above might change....

MSVS 2026 project/solution files are included with the project. While you need not use MSVS, you will need a modern C++20 compiler. See the Device Developer PDF for more information.

¹ MdbCS.java is sample code intended to be modified by users so it's technically not supported by Microchip. To avoid modifying the sample code, my changes are confined to QMdbCS.java.

Resources

My GitHub QSpice Repository

<https://github.com/robdunn4/QSpice>

In addition to this project, you'll find information about C-Block component concepts ("C-Block Basics"), various C-Block component examples, tips on using VSCode and Microsoft Visual Studio (MSVS) IDEs for C-Block development and debugging, and links to other useful QSpice-related sites/repositories.

Microchip

[MPLabX IDE](#) (free)

[Microchip MPLabX SDK / MDBCore API](#)

[Microchip Debugger \(MDB\) User's Guide](#) (one)

[Microchip Debugger \(MDB\) User's Guide](#) (another one)

[Simulator Peripheral Support By Device](#)

Microsoft

[Visual Studio Downloads](#) (2026 Community Edition is free)

Java JDK/JRE

[Eclipse Adoptium Temurin](#) (free open source Java JDK/JRE)

Conclusion

I hope that you find this project to be useful or – at least – interesting. Please let me know if you find problems or suggestions for improving the code or this documentation.

You can find me on the QSpice Forums as @RDunn.

--robert